TRON: Targeted Rule-Verifiable Online Environments for Visual Reasoning RL

Tianze Yang* Yucheng Shi* Ruitong Sun Jingyuan Huang Ninghao Liu Jin Sun
University of Georgia

Project page: https://tron-rl.github.io/

*Equal contribution.

substrate - underlying platform

arXiv:2606.01599v1 [cs.AI] 1 Jun 2026

Reinforcement learning (RL) for visual reasoning needs scalable, verifiable, and controllable training signals. Existing visual RL post-training trains on *static* curated datasets, with fixed image-question-answer samples bounded by their collection budget. In this work, we introduce TRON (*Targeted, Rule-verifiable Online eNvironments*), an online environment substrate: a training rollout is generated on demand by a controllable generator–verifier program that samples a fresh latent visual state, renders an image, asks a question, and exactly verifies the answer. A single run can therefore draw an unbounded stream of fresh instances at the difficulty level required by the current curriculum. The current TRON suite contains 520 environments organized into five ability buckets (spatial, mathematical, diagram, pattern/logic, and counting); the same substrate supports both a single *full* model trained on all buckets and per-bucket *ability-specialist* models, with no additional data collection. We also introduce a substrate analysis covering generation reliability, instance and level diversity, cross-environment near-duplicates, and base-model pass rate by difficulty level. RL post-training with TRON-DAPO consistently improves performance on ten external multimodal reasoning benchmarks across Qwen3-VL-4B, Qwen2.5-VL-7B, and MiMo-VL-7B.

1. Introduction

Recent multimodal language models increasingly rely on reward-based post-training. In domains such as reasoning, mathematics and code, RL can often exploit exact supervision: a numeric answer can be deterministically checked [11, 16, 30], a program can be executed against hidden tests [13, 19], and a proof can be machine-verified by a kernel [39, 49]. However, visual reasoning is substantially harder. A model may need to count occluded objects, infer spatial relations, trace diagrams, interpret charts, or solve visually grounded puzzles. These tasks are easy to package as evaluation examples [29, 36, 38, 46], but difficult to turn into scalable RL training signals, because each instance needs a visual scene, an unambiguous question, a calibrated difficulty, and a reliable verifier.

Existing visual reasoning training pipelines rely on image-question-answer datasets, collected either through human annotation [24, 29, 46] or synthetic instruction generation [10, 12, 23, 32, 47]. This abstraction works for evaluation but is a weak fit for RL training. First, static datasets are finite and costly to annotate, so the dataset size is bounded by the curation budget rather than by what the model could productively consume. Second, it provides little control over the specific skill being practiced or the difficulty presented to the model at different stages of training. Third, as newer VLMs absorb many popular reasoning datasets during pretraining and supervised fine-tuning, these datasets become less useful as RL training signals because the model has often already seen substantial portions of them.

We therefore take a different approach: visual reasoning RL should train on a **diverse suite of procedural environments** rather than on a fixed collection of static VQA examples. We propose TRON (Targeted, Rule-verifiable Online eNvironments), an online visual reasoning substrate in which each environment generates fresh training instances together with exact rewards. A TRON environment owns both a generator and a verifier. The generator samples a latent visual state, renders an image, and constructs a question; the verifier computes and checks the correct answer from the same state. During RL training, the model observes only the image and the question, while the reward is provided

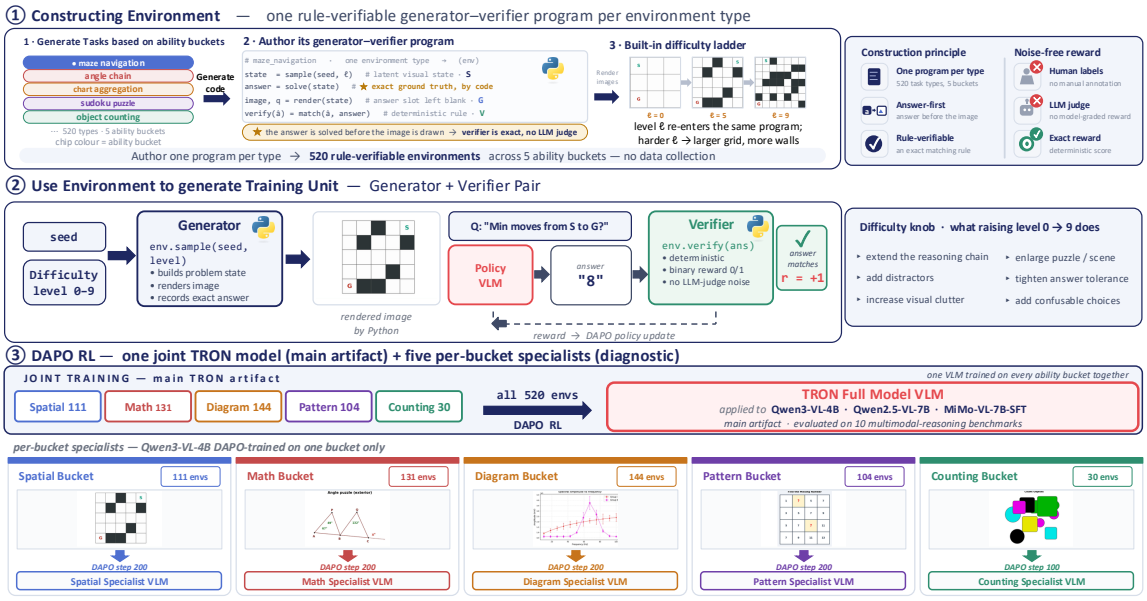


Figure 1: **TRON: diverse, ability-targeted, auditable environments for visual reasoning RL.** TRON organizes 520 rule-verifiable generators into ability buckets covering spatial, mathematical, diagram, pattern, and counting skills. Each environment produces fresh difficulty-controlled image–question rollouts with a deterministic verifier; a substrate analysis (Section 5.1) checks generation quality, instance and level diversity, cross-environment near-duplicates, and base-model pass rate by difficulty level before mixed or ability-specialist RL training.

by the deterministic verifier. This formulation allows the training process to target specific reasoning mechanisms directly. For operations such as chart aggregation, cube rotation, occluded counting, visual analogy, or graph search, we instantiate task environments centered on those operations.

This procedural formulation turns data generation into a controllable mechanism. The **diversity** of generated data is built on three levels. First, different environments target different reasoning mechanisms. Second, each environment generates distinct visual instances by varying layouts, objects, and distractors. Third, each environment has a difficulty ladder that produces progressively harder versions of the same operation. As the model improves, the training source does not become exhausted. The environments continue to generate fresh rollouts at an appropriate level of challenge.

However, generating large numbers of samples does not automatically produce useful training signal. For example, a generator may vary superficial metadata while leaving the underlying task unchanged, different environments may collapse to the same effective template, or a verifier may incorrectly accept invalid answers. TRON therefore couples controllable generation with a substrate audit that checks rendering and verifier correctness, measures diversity across instances and difficulty levels, detects near-duplicate environments, and verifies that higher difficulty levels correspond to genuinely harder tasks for the base model.

This paper makes the following contributions:

1. We introduce TRON, an *online* environment substrate for visual reasoning RL: 520 generator-verifier programs that produce fresh image-question rollouts at training time, with no cap on instances per run.
2. We organize the substrate into five ability buckets, and use it to train both a single *full* model and per-bucket *ability-specialist* models without additional data collection. Our analysis reveals new insights on ability transfer.
3. We evaluate generation quality, diversity, and difficulty calibration, showing that RL training with TRON consistently improves different open VLM families, including Qwen3-VL-4B-Instruct [1], Qwen2.5-VL-7B-Instruct [2], and MiMo-VL-7B-SFT [43], across ten reasoning benchmarks.

2. Related Work

RLVR and visual RL post-training. Reinforcement learning with verifiable rewards (RLVR) has become a central recipe for improving language-model reasoning [11, 16, 30], with variants such as GRPO and DAPO refining the optimization [30, 41]. A parallel line extends RLVR to vision-language models on mathematical, spatial, grounding, counting, and multimodal reasoning tasks [5, 14, 17, 21, 25, 28, 31, 34, 35, 40]. These methods share our use of rule-based rewards but train on *static* curated data; TRON replaces this static substrate with online generator–verifier environments that produce fresh instances and advance a per-environment curriculum on demand.

Procedural generator–verifier environments. Procedural generation has long been used to improve RL generalization [8]. Text-only systems such as Reasoning Gym, SynLogic, Enigmata, and the curriculum-driven RLVE framework [4, 18, 20, 33, 44] show that procedurally generated rule-verifiable tasks provide scalable RL signal beyond fixed math corpora, and the same principle underlies code and formal-math verifiers [13, 19, 39, 49]. Meng et al. [26] extends procedural environments to the visual setting via agentic interaction over a small set of tasks; TRON instead provides a much more diverse, image visual reasoning suite of 520 environments, each rendering images grounded in latent visual states.

Synthetic visual reasoning and capability decomposition. Synthetic visual reasoning benchmarks demonstrate the value of controlled visual states and explicit reasoning programs: CLEVR introduces functional-program supervision for counting, comparison, existence, and spatial relations [15]; RAVEN and procedurally generated matrices study visual analogy and abstract pattern completion [3, 45]; Bongard and ARC-style tasks emphasize rule induction and skill acquisition [7, 27]. Contemporary multimodal benchmarks further decompose VLM performance into mathematical, spatial, chart, diagram, logic, and puzzle-oriented capabilities [6, 24, 29, 36–38, 42, 46, 50]. TRON draws on this decomposition as an authoring guide but builds a reusable RL training suite rather than another evaluation taxonomy: 520 environments generate fresh image–question rollouts with deterministic verifiers, local difficulty ladders, and the substrate analysis of Section 5.1.

3. TRON Environments

This section presents the TRON framework: Section 3.1 defines an environment as a generator–verifier pair that produces noise-free RL signals, and Section 3.2 organizes 520 environments into five reasoning categories with per-environment difficulty ladders.

3.1 Environment Definition

A TRON environment e is a tuple $(\mathcal{S}, \mathcal{L}, G, V)$, where $\mathcal{S}$ is an *underlying task state* used to generate problem instances (e.g. cube configurations, chart data tables, or puzzle solutions), and $\mathcal{L} = \{0, 1, \ldots, 9\}$ is a set of *difficulty levels* with higher ℓ producing harder instances of the same underlying mechanism. The *generator* $G : \mathcal{S} \times \mathcal{L} \to \mathcal{I} \times \mathcal{Q} \times \mathcal{A}$ deterministically maps a sampled state $s \in \mathcal{S}$ and a level $\ell \in \mathcal{L}$ to a rendered image $I \in \mathcal{I}$, a natural-language question $q \in \mathcal{Q}$, and the ground-truth answer $a \in \mathcal{A}$. The *verifier* $V : \mathcal{A} \times \mathcal{A} \to \mathbb{R}$ takes the ground-truth a and a model prediction $\tilde{a} \in \mathcal{A}$ and returns a scalar score that reflects correctness under the environment's matching rule (exact comparison, set or sequence equality, or a puzzle-specific solver) together with any optional format signal. At training time the policy observes only (I, q) and receives reward $V(a, \tilde{a})$.

Three properties distinguish our formulation from the static image–question–answer triples used by prior visual RL datasets: 1) Dataset size is bounded only by training compute rather than by a curation budget; 2) No fixed item set exists for a successor VLM to absorb during pretraining or SFT, and 3) The difficulty parameter ℓ is a curriculum-controlled input set per *sample* rather than a set-and-forget artifact.

Bucket	#	Core mechanisms
Spatial	111	3D rotation, cube nets and folding, navigation and pathfinding, perspective shifts, mechanical layout
Math	131	Geometric theorems (angles, circles, polygons), analytic geometry, algebra, probability over visual quantities
Diagram	144	Chart aggregation (bar/line/scatter), tables, graph algorithms (trees, Eulerian/Hamiltonian, max-flow), flowcharts, scientific figures
Pattern/Logic	104	Constraint puzzles (sudoku, binairo, calcudoku), visual analogies, syllogistic deduction, sequence completion, state-space planning (Hanoi, Sokoban)
Counting	30	Visual enumeration of objects, cells, and regions; path counting; measurement and feature estimation

Table 1: Compact overview of the 520-environment TRON suite. Each bucket groups rule-verifiable generator–verifier programs around reusable visual reasoning mechanisms; Appendix A provides the fine-grained environment map.

3.2 Environment Construction

Each of the 520 TRON environments is a Python program targeting one class of reasoning mechanism. Our choice of axes follows from a survey of contemporary visual-reasoning benchmarks and datasets [6, 22, 36–38, 42], from which we distil the core abilities that strong large vision-language models are expected to handle reliably; the suite covers five such high-level ability axes (spatial, math, diagram, pattern/logic, counting; see Table 1 and Appendix A).

Producing one training instance from environment env at level ℓ takes five steps: (i) the (env, ℓ) pair determines a specific problem type — for the simplest angle-chase level, the type is "two interior angles of a triangle are given, find the third"; (ii) sample the type's free variables from a random seed (here, the two known angles, e.g. $a = 55^\circ$ and $b = 70^\circ$); (iii) apply a pre-defined formula or solver to compute the answer (here, $c = 180^\circ - a - b = 55^\circ$); (iv) render the problem into an image with the answer slot left blank (here, a triangle with a and b labelled and the third corner marked "$x = ?$"); (v) sample the question wording from a small pool of paraphrases. Because the answer is fixed before the image is drawn, the verifier always holds the unique correct value and never needs to parse the rendered image, so the RL reward is exact.

The level ℓ switches the mechanism inside the same environment: in angle-chase, ℓ scales the number of geometric deduction steps (one-step triangle sum at $\ell = 0$; four-step composite chain over parallels at $\ell = 9$); in chart-aggregation, ℓ scales the number of series, the number of time points, and whether series are stacked.

Diversity therefore enters at three places: within an environment the seed randomises the latent state (values, layout, palette, label names) and the question stem; across levels ℓ shifts the mechanism; across environments the mechanism class itself changes. Section 5.1 audits all 3 dimensions, and Appendix B shows sampled rendered examples.

4. RL Post-Training

Data generation. Training data is produced online from the TRON substrate rather than from a fixed corpus. At every sampling step the trainer picks a tuple $(\text{env}, \ell, \text{seed})$ and invokes the environment's recipe of Section 3.2 to obtain a fresh (I, q, a); because the seed is fresh each call, no two training steps see the same instance. At training time the image I is additionally perturbed to improve robustness: every sample receives a small white-pad size jitter (0–40 pixels per side), and with probability 0.30 one perturbation is drawn from {rotation ± 3–8°, low-quality JPEG, brightness shift, Gaussian blur, additive Gaussian noise}. Each environment carries its own difficulty level ℓ, which, following Zeng et al. [44], we couple to the rollout stream rather than to offline epochs: we track recent verifier accuracy at the

current ℓ and promote the environment to $\ell{+}1$ once that accuracy crosses a threshold over a target number of graded trajectories, while a sliding window over recent levels retains lower-level skills. The same 520-environment substrate supports both a single *full* model trained across all buckets and per-bucket *ability-specialist* models (Section 5.4) via a sampler configuration switch, with no extra data.

Training recipe. We optimize with a DAPO-style objective [41] and prompt-grouped advantages in the spirit of GRPO [30]: the rollout engine draws $n = 8$ responses for each prompt, the environment's deterministic verifier scores each response $\tilde{a}$, and the RL reward is $V(a, \tilde{a})$. The reward is computed without an LLM judge; each verifier checks the answer and the requested wrapper format, with lightweight numeric or symbolic normalization handled inside the verifier when appropriate. We use DAPO clip-higher (low/high clip ratio $0.2/0.28$) for exploration on negative samples and group filtering to drop prompt groups that are all-correct or all-wrong from the policy update; group filtering happens *after* verifier scoring, so uninformative rollouts still contribute their signal to the per-environment curriculum accumulator. KL regularization uses a low-variance estimator with coefficient 0.005 and is not added to the reward, and the entropy coefficient is set to 0. All runs are trained on a single node of $4\times$ H100 80 GB GPUs with vLLM tensor parallelism 4. Full hyperparameters (batch sizes, learning rate, and curriculum thresholds) for the full and ability-specialist runs are listed in Appendices C and D.

5. Experiments

We first audit the environments in TRON (Sec. 5.1) on quality, difficulty, and diversity. Then we show the main results of training state-of-the-art visual reasoning models on TRON and evaluate on external benchmarks (Sec. 5.2 and 5.3). Finally, we present ability-specialist results (Sec. 5.4), utilizing the ability partition in TRON.

5.1 Environment Analysis

We empirically validate the 520-environment substrate before any RL training. Three silent failure modes would compromise it: (i) the generator–verifier pair emits a malformed probe (blank render, missing fields, dropped sample) or admits a wrong answer, corrupting the per-sample reward; (ii) raising the difficulty level does not actually make the problem harder for the model, leaving the curriculum without a real axis; (iii) seeds within an environment produce nearly identical samples, or two distinctly-named environments collapse to nearly the same image–question distribution. We address these with the measurements summarized in Figure 2. The audit samples four levels $\ell \in \{0, 3, 6, 9\}$ and four seeds per level (8,320 probes) and grades each environment A/B/C/D. For mode (i), a quality score $Q(e)$ checks generation success, render validity, field presence, and verifier sanity. For mode (ii), which is a semantic property the static audit cannot test directly, we additionally run the Qwen3-VL-4B on the same four levels with ten seeds per level and read off its pass-rate curve (panel c). For mode (iii), a diversity

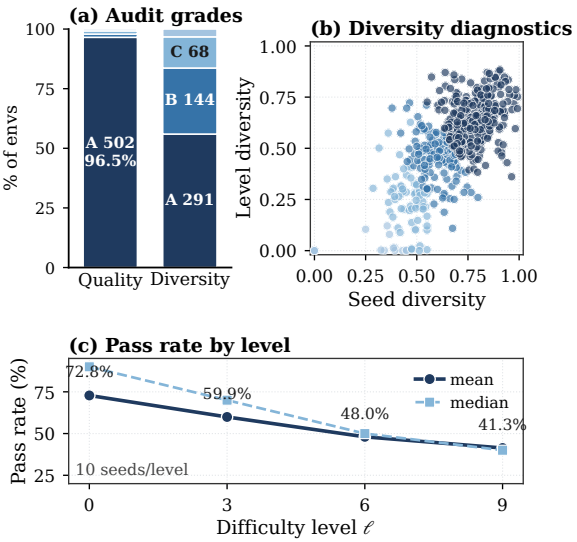


Figure 2: Model-free audit of the 520 training environments (4 levels $\times$ 4 seeds = 8,320 probes, 99.1% success). (a) Quality and diversity grade distributions. (b) Per-environment seed vs. level diversity, colored by overall diversity grade. (c) Qwen3-VL-4B base pass rate on the same audited levels.

Model	Run	Mathematical reasoning				Spatial and logical reasoning				Charts, figures, puzzles			Mean
		WeMath-S	WeMath-L	MathV	Dyna	MME-R	Spat.	Logic	HELIX	CharXiv	ChartQA	Puzzle	
Qwen3 4B	Base	52.86	70.95	43.15	66.11	42.85	77.52	57.05	21.67	39.80	34.41	72.35	52.61
	+TRON	58.29	74.86	44.54	67.49	45.88	80.47	59.73	25.56	42.40	35.04	73.25	55.23
	Δ	+5.43	+3.91	+1.39	+1.38	+3.03	+2.95	+2.68	+3.89	+2.60	+0.63	+0.90	**+2.62**
Qwen2.5 7B	Base	36.10	55.71	43.55	53.55	26.60	57.67	46.98	4.44	37.40	40.50	46.80	40.85
	+TRON	39.24	58.48	46.50	55.35	28.62	62.03	47.43	5.16	38.90	42.64	52.55	43.35
	Δ	+3.14	+2.77	+2.95	+1.80	+2.02	+4.36	+0.45	+0.72	+1.50	+2.14	+5.75	**+2.50**
MiMo 7B	Base	62.10	80.57	70.89	74.37	45.29	78.77	63.31	26.19	58.70	59.65[1]	77.20	63.37
	+TRON	68.86	82.19	73.65	76.23	46.46	86.58	66.89	30.32	62.60	60.34[1]	77.35	66.50
	Δ	+6.76	+1.62	+2.76	+1.86	+1.17	+7.81	+3.58	+4.13	+3.90	+0.69	+0.15	**+3.13**

Model abbreviations: Qwen3 4B = Qwen3-VL-4B-Instruct; Qwen2.5 7B = Qwen2.5-VL-7B-Instruct; MiMo 7B = MiMo-VL-7B-SFT.
[1] Format-normalized ChartQA Pro rejudge for MiMo; raw MiMo outputs do not match the benchmark parser.

Table 2: Main result: training on TRON consistently improves three SOTA VLM models on all external benchmarks.

score $D(e)$ combines intra-environment spread components (across seeds and across levels) with a cross-environment near-duplicate predicate. We describe $Q(e)$ and $D(e)$ below.

Quality score. Let $\mathcal{P}_e$ be the requested probes and $\mathcal{O}_e \subseteq \mathcal{P}_e$ the successfully generated ones. $Q(e)$ is the worst case across four pass rates: generation success $r_{\text{gen}} = |\mathcal{O}_e|/|\mathcal{P}_e|$, valid rendering r_{img} (fraction of $\mathcal{O}_e$ with non-trivial image size and foreground content), non-empty question and answer fields r_{qa} (fraction of $\mathcal{O}_e$ with both fields populated), and verifier sanity r_{ver} (fraction of $\mathcal{O}_e$ where the wrapped correct answer is accepted and a fixed wrong payload is rejected):

$$Q(e) = \min\{r_{\text{gen}},\, r_{\text{img}},\, r_{\text{qa}},\, r_{\text{ver}}\}. \tag{1}$$

Thresholds $\{0.98, 0.90, 0.75\}$ assign grades A/B/C, with D below. Generation succeeds for 99.07% of probes; $502/520$ environments (96.5%) receive grade A and the 18 below-A environments were re-authored. Gate predicates are in Appendix E.

Difficulty. Figure 2(c) confirms that the difficulty axis is valid for the base model: running Qwen3-VL-4B-Instruct on the same four levels with ten seeds per level, the mean pass rate across the suite falls from 72.8% at $\ell = 0$ to 59.9% ($\ell = 3$), 48.0% ($\ell = 6$), and 41.3% ($\ell = 9$), with the median following the same shape. The ≈ 31 pp drop shows that higher levels are not merely nominally relabeled but actually present harder problems, so the curriculum has a real axis to advance along.

Diversity score. $D(e) = w_s D_s(e) + w_l D_l(e) + w_x D_x(e)$ scores diversity along three axes. *Seed* diversity D_s aggregates same-level perceptual-hash (pHash) spread, question-template fraction, and answer entropy, and checks whether seeds at one level yield visibly different problems; low values flag near-identical seeds. *Level* diversity D_l aggregates cross-level pHash distance, template-Jaccard distance, and foreground-complexity shift over adjacent audited pairs, and checks whether higher levels present materially different inputs; low values flag knobs that change only hidden metadata. *Cross-environment* diversity $D_x \in \{0, 1\}$ is a near-duplicate indicator combining pHash, thumbnail, and prompt-template similarity, and flags environments that collapse to a near-duplicate of another. Appendix F gives the explicit formulas, combination weights w_s, w_l, w_x, and A–D grade thresholds. In the sweep, $435/520$ environments (83.7%) receive grade A or B; the suite-wide medians of D_s and D_l are 0.690 and 0.541.

5.2 Benchmarks

We evaluate on external multimodal reasoning benchmarks covering mathematics, spatial reasoning, chart understanding, scientific figures, visual puzzles, and logical reasoning. The evaluation suite includes

WeMath [29], MM-HELIX [48], MME-Reasoning [42], SpatialEval [36], LogicVista [38], CharXiv [37], MathVerse-Mini [46], DynaMath [50], PuzzleVQA [6], and ChartQA Pro [24]. All scores are percentages from VLMEvalKit [9] or official accuracy-like outputs.

5.3 Main Results

Table 2 shows that TRON training improves the average score for all three model families: Qwen3-VL-4B from 52.61 to 55.23, Qwen2.5-VL-7B from 40.85 to 43.35, and MiMo-VL-7B-SFT from 63.37 to 66.50. The backbones differ in pretraining recipe, generation style, and starting strength, so the consistent gains are unlikely to be an artifact of one family or one evaluation format.

The gains are not concentrated in one benchmark type, evidence that environment diversity matters. TRON improves each backbone across multiple benchmark families covering symbolic and geometric calculation, spatial reasoning, logical inference, and algorithmic visual puzzles, suggesting that the environments transfer through practiced operations rather than memorized benchmark templates.

The strongest relative pattern is on structured-reasoning benchmarks: MM-HELIX improves for every backbone, and SpatialEval gains substantially for both Qwen2.5-VL-7B and MiMo-VL-7B-SFT. These tasks align with the mechanisms TRON emphasizes (deterministic state transitions, grid or graph structure, geometric constraints, exact answer checking). Not every column moves equally; the ability-specialist analysis below shows that transfer is better explained by the underlying capability than by the benchmark category.

MiMo-VL-7B-SFT is the strongest starting point yet gains the most in mean score, suggesting that rule-verifiable RL contributes signal complementary to supervised long-reasoning tuning.

5.4 Ability-Specialist Results

Visual format vs. underlying capability. To analyze the specialist results, we first separate two properties of a VQA problem. Its *visual format* is the surface category of the image, identifiable without solving the task: charts and bar plots fall under "chart/diagram", line drawings with labelled angles and segments under "math", and so on. Its *underlying capability* is the computation a correct solver must actually perform on that image to produce the answer. The underlying capability can diverge from what the format suggests: a geometry problem may rely more on value reading than on theorem chaining, and a chart question may rely more on multi-step inference than on chart parsing. Both TRON buckets and external benchmark subtasks are classified primarily by visual format, so any specialist analysis must distinguish that format axis from the capability the problem actually demands. Each TRON environment is constructed around a single core ability, whereas a real evaluation task often combines multiple capabilities.

We organize the analysis around three questions: **RQ1:** Can a visual-format-defined bucket effectively train its specialist on subtasks within its domain? **RQ2:** Does the underlying capability transfer across visual formats? **RQ3:** Is visual-format alignment alone sufficient when the underlying capability does not match?

Each specialist starts from Qwen3-VL-4B-Instruct and uses the DAPO RL recipe, but samples from only one bucket: Math, Spatial, Count, Pattern, or Diagram. Appendix D details the setup.

RQ1 (within-bucket alignment). Since each bucket is defined by a visual format, naturally the first question is whether training on such a bucket actually improves subtasks in external benchmarks that share the same visual format. Table 3 answers this by reporting, for each specialist, representative gains on external benchmark subtasks within its bucket's domain: geometry subtasks for Math, path-planning and position subtasks for Spatial, counting subtasks for Count, and so on. Each specialist substantially improves these subtasks: Math gains $+11.2$ on WeMath angles/length, Spatial $+16.7$ on MM-HELIX maze, Count $+10.0$ on MM-HELIX hills/valleys, with similar improvements across all five buckets. Takeaway: visual-format alignment within a bucket has a clear effect: each specialist substantially improves external subtasks whose visual format matches its bucket.

Spec.	Practiced ability	Benchmark/subtask	Score	Δ
Math	Visual geometry	WeMath angles/length	39.8→**51.0**	+11.2
	Coord. geometry	WeMath coord./pos.	76.7→**83.3**	+6.7
	Arith. puzzle	MM-HELIX 24Points	83.3→**90.0**	+6.7
Spatial	Path planning	MM-HELIX maze	36.7→**53.3**	+16.7
	Position reason.	WeMath position	63.1→**73.3**	+10.3
	Spatial inference	LogicVista spatial	20.5→**29.5**	+9.0
Count	Extrema count	MM-HELIX hills/val.	26.7→**36.7**	+10.0
	Chart numeracy	CharXiv num.-in-chart	44.8→**47.8**	+3.0
	General numeracy	CharXiv num.-in-gen.	31.9→**33.2**	+1.3
Pattern	Constraint puzzle	MM-HELIX tapa	13.3→**20.0**	+6.7
	Pattern analysis	MME-R pattern	38.2→**40.3**	+2.1
	Inductive reason.	LogicVista inductive	29.9→**31.8**	+1.9
Diagram	Chart text read	CharXiv text-in-chart	39.1→**41.8**	+2.7
	Stat. visuals	DynaMath statistics	72.2→**74.6**	+2.5
	Hypothet. chart	ChartQAPro hypothet.	37.3→**39.3**	+2.0

Table 3: Within-bucket gains: each specialist on external benchmark subtasks whose visual format matches its training bucket. Δ is Spec. minus Base.

Spec.	Adjacent ability	Benchmark/subtask	Score	Δ
Math	Grid traversal	MM-HELIX maze	36.7→**56.7**	+20.0
	Coord. position	WeMath position	63.1→**74.7**	+11.6
	Spatial inference	LogicVista spatial	20.5→**32.1**	+11.5
Spatial	Angle measure	WeMath angles/length	39.8→**52.5**	+12.6
	Map navigation	WeMath route map	66.8→**73.9**	+7.1
	3D volume	MathVerse volume	32.2→**37.3**	+5.1
Count	Direction sense	WeMath direction	89.5→**100.0**	+10.5
	Unit 3D volume	MathVerse volume	32.2→**40.0**	+7.8
	Graph reasoning	DynaMath graph theory	59.6→**63.7**	+4.2
Pattern	Path constraint	MM-HELIX Hamilton	20.0→**26.7**	+6.7
	Cyclic pattern	PuzzleVQA size cycle	26.0→**32.0**	+6.0
	Chart numeracy	CharXiv num.-in-chart	44.8→**48.3**	+3.4
Diagram	Numeric pattern	PuzzleVQA rect-height	31.0→**41.0**	+10.0
	Visual numeracy	CharXiv num.-in-gen.	31.9→**38.9**	+7.0
	Struct. induction	LogicVista inductive	29.9→**35.5**	+5.6

Table 4: Cross-bucket gains: each specialist on external benchmark subtasks whose visual format lies outside its training bucket. Δ is Spec. minus Base.

Run	Mathematical reasoning				Spatial and logical reasoning				Charts, figures, puzzles			Mean
	WeMath-S	WeMath-L	MathV	Dyna	MME-R	Spat.	Logic	HELIX	CharXiv	ChartQA	Puzzle	
Base	52.86	70.95	43.15	66.11	42.85	77.52	57.05	21.67	39.80	34.41	<u>72.35</u>	52.61
Math	**58.76**	**75.90**	41.22	66.25	44.70	77.61	59.51	**26.90**	<u>42.00</u>	**35.52**	70.45	54.44
Spatial	56.38	<u>75.62</u>	42.56	65.41	44.11	78.75	59.06	23.65	39.30	**35.52**	71.85	53.84
Count	57.14	74.29	44.14	66.27	<u>45.20</u>	77.52	59.28	23.97	<u>42.00</u>	34.08	69.90	53.98
Pattern	54.48	71.81	41.57	65.33	43.27	76.22	56.38	23.49	41.60	34.80	71.45	52.76
Diagram	57.81	74.76	**44.67**	<u>66.69</u>	44.70	<u>79.01</u>	**60.18**	25.40	41.90	33.94	71.40	<u>54.59</u>
Full	<u>58.29</u>	74.86	<u>44.54</u>	**67.49**	**45.88**	**80.47**	<u>59.73</u>	<u>25.56</u>	**42.40**	<u>35.04</u>	**73.25**	**55.23**

Table 5: Broad-suite effect of ability specialists. Each specialist trains on one ability bucket; Full is the jointly trained model. Bold marks the best score per benchmark or mean, and underlining marks the second-best score.

RQ2 (cross-bucket carryover). Table 4 shows that each specialist also lifts external subtasks outside its bucket, where the visual format differs from its training but a shared underlying capability remains.

The most dramatic case is Math on MM-HELIX maze (+20.0), alongside WeMath position (+11.6) and LogicVista spatial (+11.5), consistent with Math training a transferable multi-step reasoning capability rather than a maze-specific skill. Spatial transfers to WeMath angles/length (+12.6) and route map (+7.1), consistent with a shared spatial-understanding capability across formats. Count transfers to MathVerse volume (+7.8) and DynaMath graph theory (+4.2), consistent with a visual-grounding capability over discrete elements. Pattern transfers to MM-HELIX Hamiltonian path (+6.7), consistent with constraint and rule reasoning over discrete visual structures. Diagram's bar-chart training transfers to PuzzleVQA rect-height (+10.0), consistent with a figure-reading capability. Takeaway: as long as the underlying capability a task requires has been trained, that capability can transfer to the task even when the task's visual format does not appear in training.

RQ3 (visual format alone). Table 5 reports each specialist's accuracy across the 10 external benchmarks. The visually-aligned specialist (whose bucket shares the benchmark's visual format) wins only on WeMath; two cases show why. On MathVerse, three of the five problem versions strip the textual description, so the bottleneck is figure-reading across structured geometric visuals rather than theorem chaining: Diagram trains figure-reading and wins, while Math bundles figure-reading inside theorem chains where it never becomes a standalone skill, and ends up regressing below Base. CharXiv (Reasoning split) is the opposite: a multi-step reasoning load over scientific charts on which Diagram falls behind Math because the questions require compositional parsing, ordinal comparison, and chained inference; Math trains exactly these dense inference chains, while Diagram's chart extraction and aggregation are too shallow. Takeaway: both visual format and underlying capability contribute to task improvement (RQ1 and RQ2), but visual-format alignment alone is not sufficient: the visually-aligned specialist wins only 1 of 10 benchmarks, so an effective training set should cover both the format and the underlying capabilities a task demands.

6. Discussion

Live environments preserve freshness and an advancing curriculum. Parametric live environments give every sampled instance an exact reward, supply fresh latent states and renderings that keep memorization pressure low, and expose a per-environment difficulty ladder the curriculum can advance on demand. Pre-generating TRON rollouts into a static parquet would forfeit the last two properties: the snapshot is bounded and exhausts once seen, its curriculum cannot advance after collection, and its bucket mix is frozen at generation time, so ability-specialist re-targeting becomes a re-collection effort rather than a sampler switch.

Underlying capability is the broader driver of transfer. A benchmark's visual format describes its inputs, not the underlying capabilities a model must exercise to solve it, and a single benchmark typically demands several capabilities at once. Our specialist analysis (Section 5.4) shows that single-bucket RL transfers better along the capability axis rather than the visual-format axis.

Diversity hedges against unidentified capabilities. In practice, the underlying capability of a real task is hard to decouple: a single question often interleaves several capabilities and has no canonical breakdown, so we cannot reliably identify in advance which capability a future task will demand. The simplest robust response is to make the training-environment set as diverse as possible, covering as many underlying capabilities as we can, so that whichever capability an unseen task actually demands is likely already in the mix. This is the design rationale behind TRON's 520 environments, spanning five ability buckets and, within each, as many visual formats, generation mechanisms, and parameter ranges as we could audit.

7. Conclusion

We introduced TRON, an online environment substrate for visual reasoning RL: 520 generator-verifier programs organized into five ability buckets, each producing fresh image-question rollouts with exact rule-based rewards and a local difficulty ladder. We pair the substrate with an audit that measures quality,

diversity, and difficulty before training. Across Qwen3-VL-4B, Qwen2.5-VL-7B, and MiMo-VL-7B-SFT, RL post-training with TRON-DAPO consistently improves performance on ten external multimodal reasoning benchmarks, supporting online environments as a practical substrate for visual reasoning training.

8. Limitations

TRON environments are synthetic, so their visual style and language can diverge from real benchmark data; the audit catches internal quality failures but cannot guarantee distributional alignment with every external benchmark, especially for domains needing photographic or dense scientific perception.

Difficulty levels are author-chosen generator parameters. The aggregate base-model pass rate falls monotonically across levels (Figure 2c), but individual environments need not be strictly monotone and step sizes can vary.

The diversity analysis depends on hand-chosen hyperparameters: signal normalizations, per-component weights inside D_s and D_l, dup thresholds, and the convex-combination weights (w_s, w_l, w_x) with A–D grade cutoffs (Appendix F). These were chosen to make the grade histogram informative on the current suite rather than learned, and a different operational definition of "sufficient diversity" would shift the boundaries.

The five ability buckets are not strictly decoupled: many environments exercise more than one mechanism (a chart task may also need multi-step reasoning, a graph-algorithm task may also need counting), so the labels should be read as a coarse partition rather than a clean factorization.

9. Ethical Considerations

Data and models. All training environments in TRON are generated procedurally; we do not collect or scrape any new data, and no personally identifiable information is involved. Evaluation uses public multimodal reasoning benchmarks (Section 5.2) and open-source vision-language models (Qwen3-VL-4B-Instruct, Qwen2.5-VL-7B-Instruct, and MiMo-VL-7B-SFT) under their respective licenses.

Intended use and misuse. Our work targets improving the visual reasoning capabilities of vision-language models through procedurally generated, rule-verifiable environments. It is intended for benign applications such as educational tools, assistive vision, scientific figure understanding, and automated analysis of structured visuals. The improved reasoning ability could in principle be repurposed for surveillance or other sensitive monitoring tasks; however, the contribution here is methodological (a training substrate and curriculum framework) and is not tailored toward such use cases.

Use of LLMs. Large language models were used for language polishing (grammar and phrasing) and for coding assistance, including writing and debugging portions of the procedural environment generators and analysis code. All research ideas, problem formulations, methodology, experimental design, analyses, and claims are the authors' own.

References

[1] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, et al. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025.

[2] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, Junyang Lin, et al. Qwen2.5-VL technical report. *arXiv preprint arXiv:2502.13923*, 2025.

[3] David G. T. Barrett, Felix Hill, Adam Santoro, Ari S. Morcos, and Timothy Lillicrap. Measuring abstract reasoning in neural networks. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Proceedings of Machine Learning Research*, pages 511–520, 2018.

[4] Jiangjie Chen, Qianyu He, Siyu Yuan, Aili Chen, Zhicheng Cai, Weinan Dai, Hongli Yu, Jiaze Chen, Xuefeng Li, Qiying Yu, et al. Enigmata: Scaling logical reasoning in large language models with synthetic verifiable puzzles. *Advances in Neural Information Processing Systems*, 38:3613–3661, 2026.

[5] Liang Chen, Lei Li, Haozhe Zhao, Yifan Song, and Vinci. R1-V: Reinforcing super generalization ability in vision-language models with less than $3. https://github.com/Deep-Agent/R1-V, 2025.

[6] Yew Ken Chia, Vernon Toh Yan Han, Deepanway Ghosal, Lidong Bing, and Soujanya Poria. PuzzleVQA: Diagnosing multimodal reasoning challenges of language models with abstract visual patterns. *arXiv preprint arXiv:2403.13315*, 2024.

[7] François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.

[8] Karl Cobbe, Christopher Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119, pages 2048–2056. PMLR, 2020.

[9] Haodong Duan, Junming Yang, Yuxuan Qiao, Xinyu Fang, Lin Chen, Yuan Liu, Xiaoyi Dong, Yuhang Zang, Pan Zhang, Jiaqi Wang, et al. Vlmevalkit: An open-source toolkit for evaluating large multi-modality models. In *Proceedings of the 32nd ACM international conference on multimedia*, pages 11198–11201, 2024.

[10] Jiahui Gao, Renjie Pi, Jipeng Zhang, Jiacheng Ye, Wanjun Zhong, Yufei Wang, Lanqing Hong, Jianhua Han, Hang Xu, Zhenguo Li, and Lingpeng Kong. G-LLaVA: Solving geometric problem with multi-modal large language model. *arXiv preprint arXiv:2312.11370*, 2023.

[11] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[12] Yucheng Han, Chi Zhang, Xin Chen, Xu Yang, Zhibin Wang, Gang Yu, Bin Fu, and Hanwang Zhang. ChartLlama: A multimodal LLM for chart understanding and generation. *arXiv preprint arXiv:2311.16483*, 2023.

[13] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring coding challenge competence with APPS. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021.

[14] Wenxuan Huang, Bohan Jia, Zijie Zhai, Shaosheng Cao, Zheyu Ye, Fei Zhao, Zhe Xu, Yao Hu, and Shaohui Lin. Vision-R1: Incentivizing reasoning capability in multimodal large language models. *arXiv preprint arXiv:2503.06749*, 2025.

[15] Justin Johnson, Bharath Hariharan, Laurens van der Maaten, Li Fei-Fei, C. Lawrence Zitnick, and Ross Girshick. CLEVR: A diagnostic dataset for compositional language and elementary visual reasoning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2901–2910, 2017.

[16] Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, et al. Tülu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024.

[17] Sicong Leng, Jing Wang, Jiaxi Li, Hao Zhang, Zhiqiang Hu, Boqiang Zhang, Yuming Jiang, Hang Zhang, Xin Li, Lidong Bing, et al. Mmr1: Enhancing multimodal reasoning with variance-aware sampling and open resources. *arXiv preprint arXiv:2509.21268*, 2025.

[18] Jia Li, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Huang, Kashif Rasul, Longhui Yu, Albert Q. Jiang, Ziju Shen, et al. NuminaMath: The largest public dataset in AI4Maths with 860k pairs of competition math problems and solutions. Technical report, Hugging Face / Project Numina, 2024.

[19] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022.

[20] Junteng Liu, Yuanxiang Fan, Zhuo Jiang, Han Ding, Yongyi Hu, Chi Zhang, Yiqi Shi, Shitong Weng, Aili Chen, Shiqi Chen, Yunan Huang, Mozhi Zhang, Pengyu Zhao, Junjie Yan, and Junxian He. SynLogic: Synthesizing verifiable reasoning data at scale for learning logical reasoning and beyond. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2505.19641.

[21] Ziyu Liu, Zeyi Sun, Yuhang Zang, Xiaoyi Dong, Yuhang Cao, Haodong Duan, Dahua Lin, and Jiaqi Wang. Visual-RFT: Visual reinforcement fine-tuning. *arXiv preprint arXiv:2503.01785*, 2025.

[22] Pan Lu, Hritik Bansal, Tony Xia, Jiacheng Liu, Chunyuan Li, Hannaneh Hajishirzi, Hao Cheng, Kai-Wei Chang, Michel Galley, and Jianfeng Gao. MathVista: Evaluating mathematical reasoning of foundation models in visual contexts. In *International Conference on Learning Representations (ICLR)*, 2024.

[23] Ahmed Masry, Megh Thakkar, Aayush Bajaj, Aaryaman Kartha, Enamul Hoque, and Shafiq Joty. ChartGemma: Visual instruction-tuning for chart reasoning in the wild. *arXiv preprint arXiv:2407.04172*, 2024.

[24] Ahmed Masry, Mohammed Saidul Islam, Mahir Ahmed, Aayush Bajaj, Firoz Kabir, Aaryaman Kartha, Md Tahmid Rahman Laskar, Mizanur Rahman, Shadikur Rahman, Mehrad Shahmohammadi, et al. ChartQAPro: A more diverse and challenging benchmark for chart question answering. *arXiv preprint arXiv:2504.05506*, 2025.

[25] Fanqing Meng, Lingxiao Du, Zongkai Liu, Zhixiang Zhou, Quanfeng Lu, Daocheng Fu, Tiancheng Han, Botian Shi, Wenhai Wang, Junjun He, Kaipeng Zhang, Ping Luo, Yu Qiao, Qiaosheng Zhang, and Wenqi Shao. MM-Eureka: Exploring the frontiers of multimodal reasoning with rule-based reinforcement learning. *arXiv preprint arXiv:2503.07365*, 2025.

[26] Fanqing Meng, Lingxiao Du, Jiawei Gu, Jiaqi Liao, Linjie Li, Zijian Wu, Xiangyan Liu, Ziqi Zhao, Mengkang Hu, Zichen Liu, et al. Gym-v: A unified vision environment system for agentic vision research. *arXiv preprint arXiv:2603.15432*, 2026.

[27] Weili Nie, Zhiding Yu, Lei Mao, Ankit B. Patel, Yuke Zhu, and Anima Anandkumar. Bongard-LOGO: A new benchmark for human-level concept learning and reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[28] Yingzhe Peng, Gongrui Zhang, Miaosen Zhang, Zhiyuan You, Jie Liu, Qipeng Zhu, Kai Yang, Xingzhong Xu, Xin Geng, and Xu Yang. LMM-R1: Empowering 3b LMMs with strong reasoning abilities through two-stage rule-based rl. *arXiv preprint arXiv:2503.07536*, 2025.

[29] Runqi Qiao, Qiuna Tan, Guanting Dong, Minhui Wu, Chong Sun, Xiaoshuai Song, Zhuoma GongQue, Shanglin Lei, Zhe Wei, Miaoxuan Zhang, Runfeng Qiao, Yifan Zhang, Xiao Zong, Yida Xu, Muxi Diao, Zhimin Bao, Chen Li, and Honggang Zhang. We-Math: Does your large multimodal model achieve human-like mathematical reasoning? *arXiv preprint arXiv:2407.01284*, 2024.

[30] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[31] Haozhan Shen, Peng Liu, Jingcheng Li, Chunxin Fang, Yibo Ma, Jiajia Liao, Qiaoli Shen, Zilun Zhang, Kangjia Zhao, Qianqian Zhang, Ruochen Xu, and Tiancheng Zhao. VLM-R1: A stable and generalizable r1-style large vision-language model. *arXiv preprint arXiv:2504.07615*, 2025.

[32] Wenhao Shi, Zhiqiang Hu, Yi Bin, Junhua Liu, Yang Yang, See-Kiong Ng, Lidong Bing, and Roy Ka-Wei Lee. Math-LLaVA: Bootstrapping mathematical reasoning for multimodal large language models. *arXiv preprint arXiv:2406.17294*, 2024.

[33] Zafir Stojanovski, Oliver Stanley, Joe Sharratt, Richard Jones, Abdulhakeem Adefioye, Jean Kaddour, and Andreas Köpf. Reasoning gym: Reasoning environments for reinforcement learning with verifiable rewards. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2505.24760.

[34] Huajie Tan, Yuheng Ji, Xiaoshuai Hao, Minglan Lin, Pengwei Wang, Zhongyuan Wang, and Shanghang Zhang. Reason-RFT: Reinforcement fine-tuning for visual reasoning. *arXiv preprint arXiv:2503.20752*, 2025.

[35] Haozhe Wang, Chao Qu, Zuming Huang, Wei Chu, Fangzhen Lin, and Wenhu Chen. VL-Rethinker: Incentivizing self-reflection of vision-language models with reinforcement learning. *arXiv preprint arXiv:2504.08837*, 2025.

[36] Jiayu Wang, Yifei Ming, Zhenmei Shi, Vibhav Vineet, Xin Wang, Yixuan Li, and Neel Joshi. Is a picture worth a thousand words? delving into spatial reasoning for vision language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[37] Zirui Wang, Mengzhou Xia, Luxi He, Howard Chen, Yitao Liu, Richard Zhu, Kaiqu Liang, Xindi Wu, Haotian Liu, Sadhika Malladi, Alexis Chevalier, Sanjeev Arora, and Danqi Chen. CharXiv: Charting gaps in realistic chart understanding in multimodal LLMs. *arXiv preprint arXiv:2406.18521*, 2024.

[38] Yijia Xiao, Edward Sun, Tianyu Liu, and Wei Wang. LogicVista: Multimodal LLM logical reasoning benchmark in visual contexts. *arXiv preprint arXiv:2407.04973*, 2024.

[39] Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023.

[40] Yi Yang, Xiaoxuan He, Hongkun Pan, Xiyan Jiang, Yan Deng, Xingtao Yang, Haoyu Lu, Dacheng Yin, Fengyun Rao, Minfeng Zhu, Bo Zhang, and Wei Chen. R1-Onevision: Advancing generalized multimodal reasoning through cross-modal formalization. *arXiv preprint arXiv:2503.10615*, 2025.

[41] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. DAPO: An open-source LLM reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

[42] Jiakang Yuan, Tianshuo Peng, Yilei Jiang, Yiting Lu, Renrui Zhang, Kaituo Feng, Chaoyou Fu, Tao Chen, Lei Bai, Bo Zhang, et al. MME-Reasoning: A comprehensive benchmark for logical reasoning in MLLMs. *arXiv preprint arXiv:2505.21327*, 2025.

[43] Zihao Yue, Zhenru Lin, Yifan Song, Weikun Wang, Shuhuai Ren, Shuhao Gu, Shicheng Li, Peidian Li, Liang Zhao, Lei Li, et al. Mimo-vl technical report. *arXiv preprint arXiv:2506.03569*, 2025.

[44] Zhiyuan Zeng, Hamish Ivison, Yiping Wang, Lifan Yuan, Shuyue Stella Li, Zhuorui Ye, Siting Li, Jacqueline He, Runlong Zhou, Tong Chen, et al. Rlve: Scaling up reinforcement learning for language models with adaptive verifiable environments. *arXiv preprint arXiv:2511.07317*, 2025.

[45] Chi Zhang, Feng Gao, Baoxiong Jia, Yixin Zhu, and Song-Chun Zhu. RAVEN: A dataset for relational and analogical visual rEasoNing. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5317–5327, 2019.

[46] Renrui Zhang, Dongzhi Jiang, Yichi Zhang, Haokun Lin, Ziyu Guo, Pengshuo Qiu, Aojun Zhou, Pan Lu, Kai-Wei Chang, Peng Gao, and Hongsheng Li. MathVerse: Does your multi-modal LLM truly see the diagrams in visual math problems? *arXiv preprint arXiv:2403.14624*, 2024.

[47] Renrui Zhang, Xinyu Wei, Dongzhi Jiang, Ziyu Guo, Yichi Zhang, Chengzhuo Tong, Jiaming Liu, Aojun Zhou, Shanghang Zhang, Gao Peng, et al. Mavis: Mathematical visual instruction tuning with an automatic data engine. In *International Conference on Learning Representations*, volume 2025, pages 87955–87989, 2025.

[48] Xiangyu Zhao, Junming Lin, Tianhao Liang, Yifan Zhou, Wenhao Chai, Yuzhe Gu, Weiyun Wang, Kai Chen, Gen Luo, Wenwei Zhang, Junchi Yan, Hua Yang, Haodong Duan, and Xue Yang. MM-HELIX: Boosting multimodal long-chain reflective reasoning with holistic platform and adaptive hybrid policy optimization. *arXiv preprint arXiv:2510.08540*, 2025.

[49] Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. miniF2F: A cross-system benchmark for formal olympiad-level mathematics. In *International Conference on Learning Representations (ICLR)*, 2022.

[50] Chengke Zou, Xingang Guo, Rui Yang, Junyu Zhang, Bin Hu, and Huan Zhang. DynaMath: A dynamic visual benchmark for evaluating mathematical reasoning robustness of vision language models. *arXiv preprint arXiv:2411.00836*, 2024.

A. Fine-Grained Environment Coverage

Table 6 expands the high-level suite composition in Table 1. The entries are representative rather than exhaustive; each listed environment is a generator–verifier program with multiple seeds and difficulty levels.

B. Qualitative Environment Examples

Figures 3–7 show qualitative examples sampled directly from the 520 training environments. Each page focuses on one ability bucket, with two generator families as rows and Levels 0, 5, and 9 as columns. Each panel pairs the rendered instance with its task prompt and verified answer; only repeated answer-format boilerplate is omitted for readability. The examples therefore show both sides of the environment contract: the visual instance given to the policy and the answer accepted by the executable verifier.

C. Full-Model Training Details

The three full TRON runs reported in Table 2 share the same DAPO-style training recipe, the same online environment sampler, and the same curriculum-promotion mechanism. Backbone-specific batch sizes and vLLM memory settings are adjusted to fit a four-GPU H100 80 GB node. Table 7 lists the per-backbone hyperparameters, with shared settings repeated across the three columns for readability.

Curriculum promotion rule (identical across all three runs). At each training step, every prompt generates $n = 8$ rollouts that the verifier scores 0/1. The per-environment curriculum manager maintains a sliding window of the four most recent rollout groups for the environment's current level ℓ, giving a buffer of $4 \times 8 = 32$ scored trajectories at any time. Once the mean accuracy over the buffer reaches ≥ 0.80, the environment's level is advanced from ℓ to $\ell + 1$ and the buffer is reset. Group filtering removes uninformative prompt groups (all-correct or all-wrong) from the policy update, but every scored rollout still contributes to the curriculum accumulator. The sampler keeps a 0.30 probability of mixing lower-level instances so that training does not collapse onto only the hardest level reached. Checkpoints and validation parquets are saved every 10 training steps.

D. Ability-Specialist Training Details

The ability-specialist runs use the same DAPO/GRPO training stack as the broad mixed run, but restrict the online environment sampler to one ability bucket. The launcher reads the bucket list, sets the environment filter accordingly, and writes separate checkpoints, validation generations, and curriculum state for each ability. All specialists start from Qwen3-VL-4B-Instruct and use the same rule-based reward function as the full model.

Count is capped at step 100 because it has only 30 training environments, compared with 104–144 environments for the other buckets. Running it to the same 200-step horizon would give the Count specialist disproportionately many gradient updates per environment, increasing the risk of overfitting to the small bucket; the 100-step cap keeps the per-environment update count roughly comparable to the other specialists.

For all specialists, the online training epoch size is 3200 generated prompts. The training batch, generation batch, and PPO mini-batch are all 32, with eight rollouts per prompt. Rollouts use temperature 1.0, maximum prompt length 8192, and maximum response length 8192. Training uses four GPUs with vLLM tensor parallelism 4. The actor learning rate is 5×10^{-6}, Adam betas are $(0.9, 0.98)$, entropy coefficient is 0, clipping uses the $[0.2, 0.28]$ range, and the actor KL loss coefficient is 0.005. KL is not added directly to the reward. Group filtering is enabled with accuracy as the filtering metric and at most

Bucket	Fine-grained domain	Representative environments	Reasoning operations covered
Spatial Reasoning	3D rotation and chirality	*spatial rotation*, *polycube rotation axis identify*, *shape rotation invariant*, *chiral object identification*, *chirality pair discrimination*	Mental rotation, handedness, invariant matching, and axis-based object transformation.
	Projection, folding, and cross-sections	*net folding*, *orthographic projection*, *projection view*, *solid cross section reverse*, *unfold path prediction*	Mapping between 2D views and 3D structure; inferring hidden geometry from folds, nets, and slices.
	Navigation and relative direction	*map distance*, *maze solution length*, *relative direction chain*, *bearing compass*, *map route optimization*	Route planning, shortest paths, compass direction, egocentric reference frames, and chained spatial relations.
	Object layout and viewpoint	*before after*, *depth order*, *near far mcq*, *perspective shift*, *object tracking across frames*	Viewpoint changes, occlusion order, temporal tracking, and relative object placement.
	Mechanical and physical diagrams	*lever pulley*, *gear rotation direction*, *scale balance*, *pendulum compare*, *submersion water rise*	Physical causality, force/ratio reasoning, mechanical linkage, and qualitative dynamics.
Mathematical Reasoning	Geometry theorem reasoning	*angle chain*, *angle bisector chain deep*, *circle theorem*, *secant tangent*, *cyclic quadrilateral advanced*, *ptolemy quad*	Multi-step angle chasing, circle facts, quadrilateral constraints, and theorem-backed diagram reasoning.
	Coordinate and analytic geometry	*coordinate geometry*, *analytic geom chain*, *distance between points*, *line parallel perp*, *two lines intersection*, *conic eccentricity mcq*	Reading coordinates, applying formulas, comparing slopes, solving intersections, and identifying conic properties.
	Functions and calculus from graphs	*function graph value read*, *derivative graph*, *parabola vertex*, *implicit function level set*, *area under curve estimation*, *continuity at point*	Graph-to-symbol translation, local/global function properties, rate-of-change, and visual calculus.
	Probability and statistics	*probability tree*, *spinner probability*, *conditional probability visual*, *histogram*, *box plot comparison*, *confusion matrix diagonal*	Event composition, conditional probability, distribution reading, summary statistics, and diagnostic-table reasoning.
	Algebraic and arithmetic visuals	*expression substitute evaluate*, *matrix operation*, *omitted operator*, *sliding sum blanks*, *numeric commonsense visual*, *unit conversion visual*	Symbolic substitution, matrix arithmetic, missing-operator search, numeric commonsense, and unit conversion.
Visual Diagram Understanding	Chart extraction and aggregation	*chart multistep*, *bar chart aggregate*, *chart aggregate claim*, *chart filter aggregate*, *chart kth largest*, *chart percent change*	Value lookup, aggregation, ranking, filtering, percent change, and claim verification over charts.
	Complex plots and scientific figures	*dual axis chart*, *ternary plot*, *heatmap pattern identification*, *phase diagram*, *scientific graph interpretation*, *kinematics graph*	Multi-axis reading, scientific plot interpretation, phase/region reasoning, and trend or threshold inference.
	Tables, schedules, and joins	*table cell lookup*, *business table*, *pivot table*, *multi table join*, *schedule table*, *boarding pass duration*	Cell lookup, row/column aggregation, relational joins, schedule arithmetic, and structured-record comparison.
	Flow, graph, and process diagrams	*flowchart*, *process flow diagram*, *dependency graph*, *circuit output prediction*, *food web*, *labeled parts diagram*	Following arrows and dependencies, circuit logic, process-state updates, and label-to-part grounding.
	Text, labels, and infographic reasoning	*text render math*, *handwritten expression*, *chart with latex label*, *chart with context paragraph*, *infographic business*, *ambiguous label resolution*	OCR-like reading, label disambiguation, text-plus-visual grounding, and paragraph-conditioned visual reasoning.
Visual Pattern & Logical Reasoning	Visual analogy and sequences	*visual analogy raven 3x3*, *analogy multiple dimensions*, *analogy from sequence*, *figure sequence next*, *visual sequence*, *matrix completion 5x5*	Raven-style abstraction, analogy transfer, sequence continuation, and matrix completion.
	Rule induction and classification	*rule induction sequence*, *pattern rule multi example*, *inductive rule discovery*, *visual rule exception*, *attribute enumeration discovery*, *odd one out*	Inferring latent rules, handling exceptions, attribute selection, and category comparison.
	Deductive and symbolic logic	*logic grid*, *syllogism passage*, *quantifier logic*, *truth table 3variable*, *logical negation chain*, *argument contradiction*	Constraint propagation, syllogistic reasoning, truth tables, quantifiers, negation, and contradiction detection.
	Constraint puzzles	*calcudoku*, *futoshiki solve*, *hitori solve*, *binairo solve*, *nonogram solve*, *skyscrapers solve*, *numbrix solve*	Grid constraints, local/global consistency, search, and exact puzzle verification.
	Algorithmic and graph search	*eulerian path find*, *hamiltonian path find*, *kruskal first edge*, *topological sort*, *shortest path directed weighted*, *stack queue trace*	Graph traversal, path existence, algorithm tracing, data-structure state updates, and weighted shortest paths.
Counting & Quantitative Estimation	Object and instance counting	*object count with occlusion*, *precise counting*, *visual counting ultra easy*, *shape instance count*, *dense 2d count warmup*, *missing grid count*	Counting visible and missing items, handling distractors, and separating object instances from background clutter.
	Region, cell, and path counts	*region counting*, *region count grid*, *grid cell count with rules*, *path counting*, *path length grid count*, *lattice path count*	Counting cells, regions, admissible paths, path lengths, and rule-conditioned grid structures.
	3D and stacked counts	*isometric counting*, *multi view cube count*, *hidden cube inference*, *layered stack count*, *cube decomposition count*, *polycube counting warmup*	Inferring counts from 3D views, hidden blocks, layers, decompositions, and polycube structure.
	Attribute grouping and quantization	*attribute count quantize*, *attribute height quantize*, *attribute ordering*, *feature counting classification*, *stroke continuity grouping*, *shape symmetry grouping*	Grouping by visual attributes, quantizing continuous features, ordering, and classifying by shape or stroke cues.
	Robust perception and traps	*blank annotation trap*, *chart no labels*, *chart unanswerable trap*, *mcq answer not in options*, *optical illusion lines*, *visual occlusion counting*	Detecting missing evidence, rejecting invalid options, handling unanswerable prompts, illusions, and occlusion.

Table 6: Fine-grained capability map for the TRON environment suite. This table expands Table 1 by listing the subdomains that drive environment authoring and representative generator–verifier programs used during training.

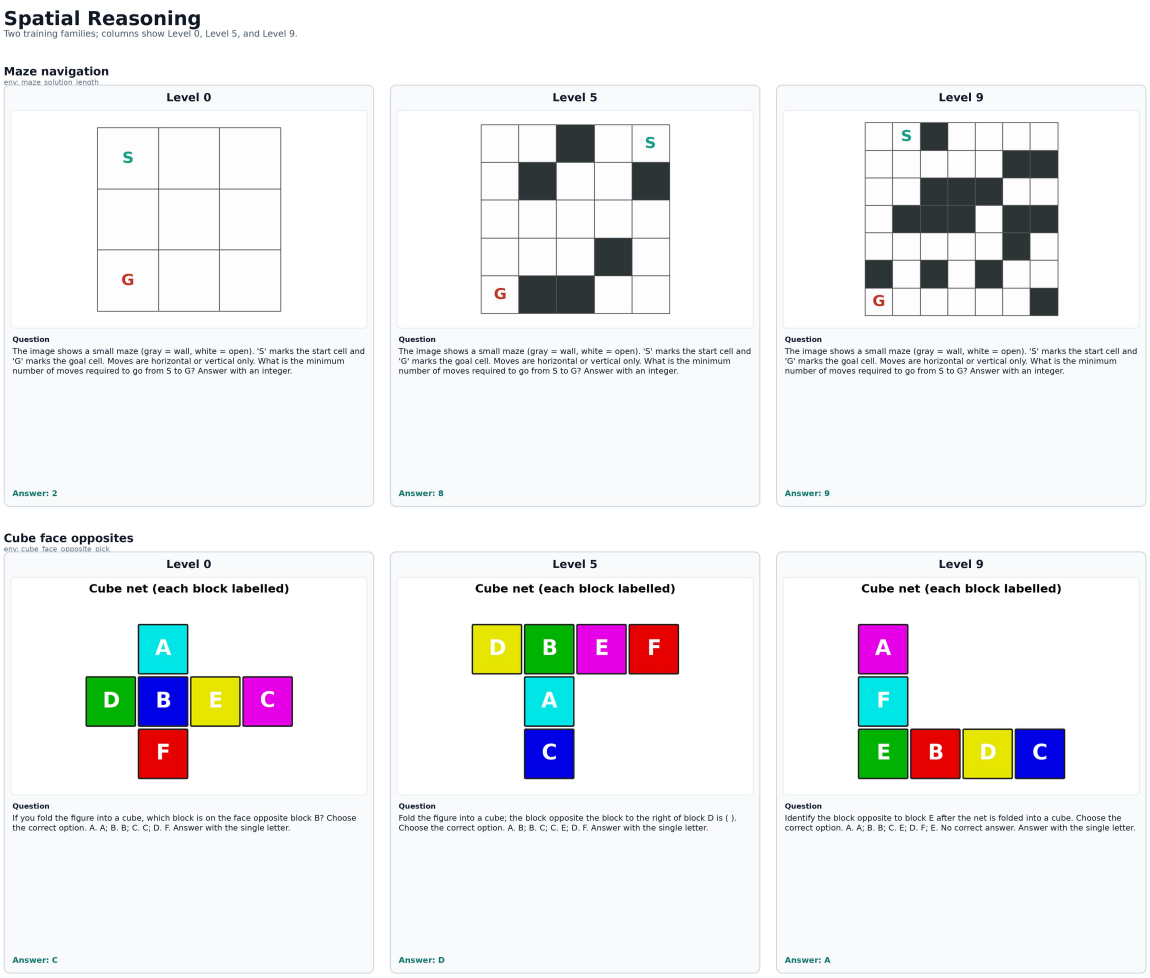


Figure 3: Spatial Reasoning examples. Rows show maze navigation and cube-net opposite-face reasoning; columns increase the difficulty level while keeping the generator family fixed.

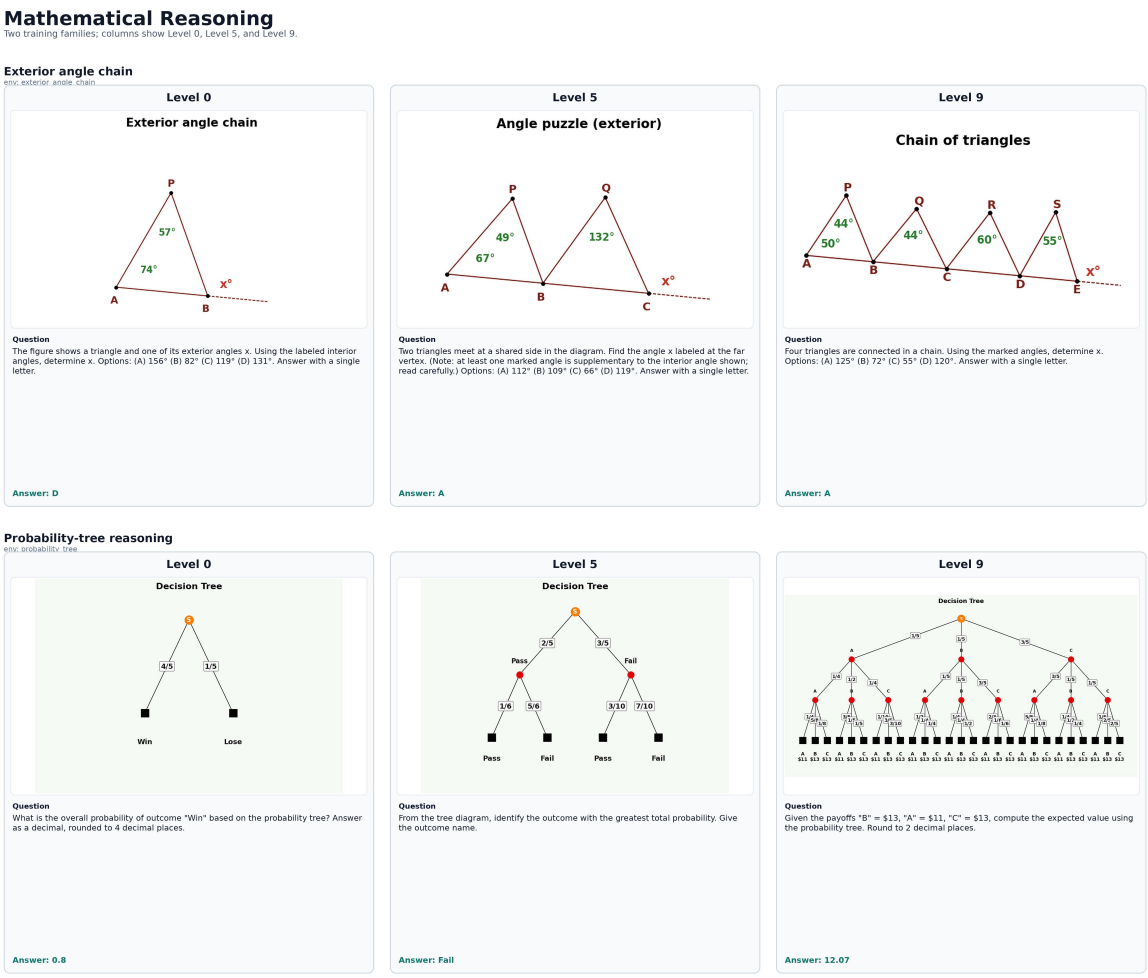


Figure 4: Mathematical Reasoning examples. Rows show exterior-angle geometry and probability-tree reasoning. Difficulty increases through longer angle chains, denser trees, and more compositional numerical queries.

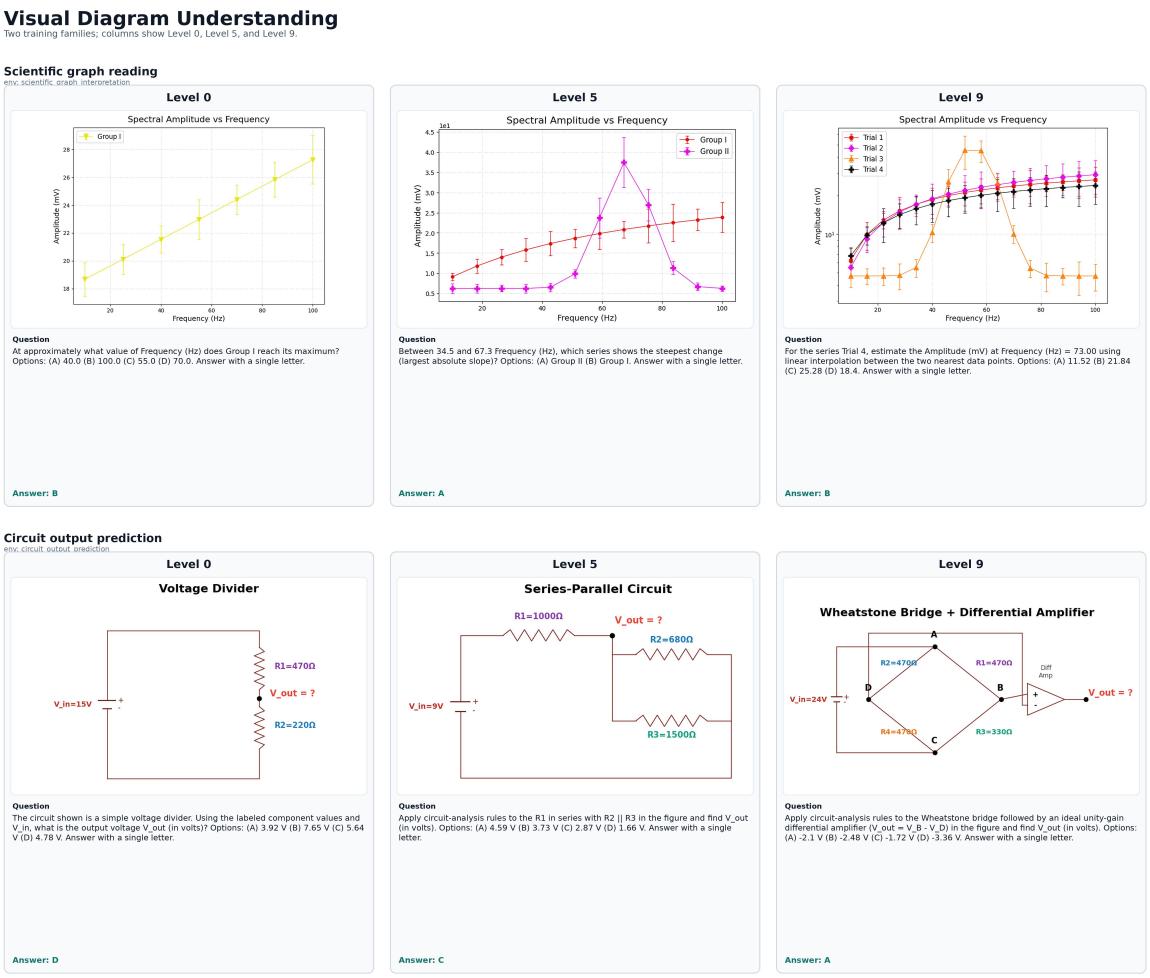


Figure 5: Visual Diagram Understanding examples. Rows show scientific graph interpretation and circuit output prediction. Higher levels add more plotted series, interpolation, and more complex circuit topology.

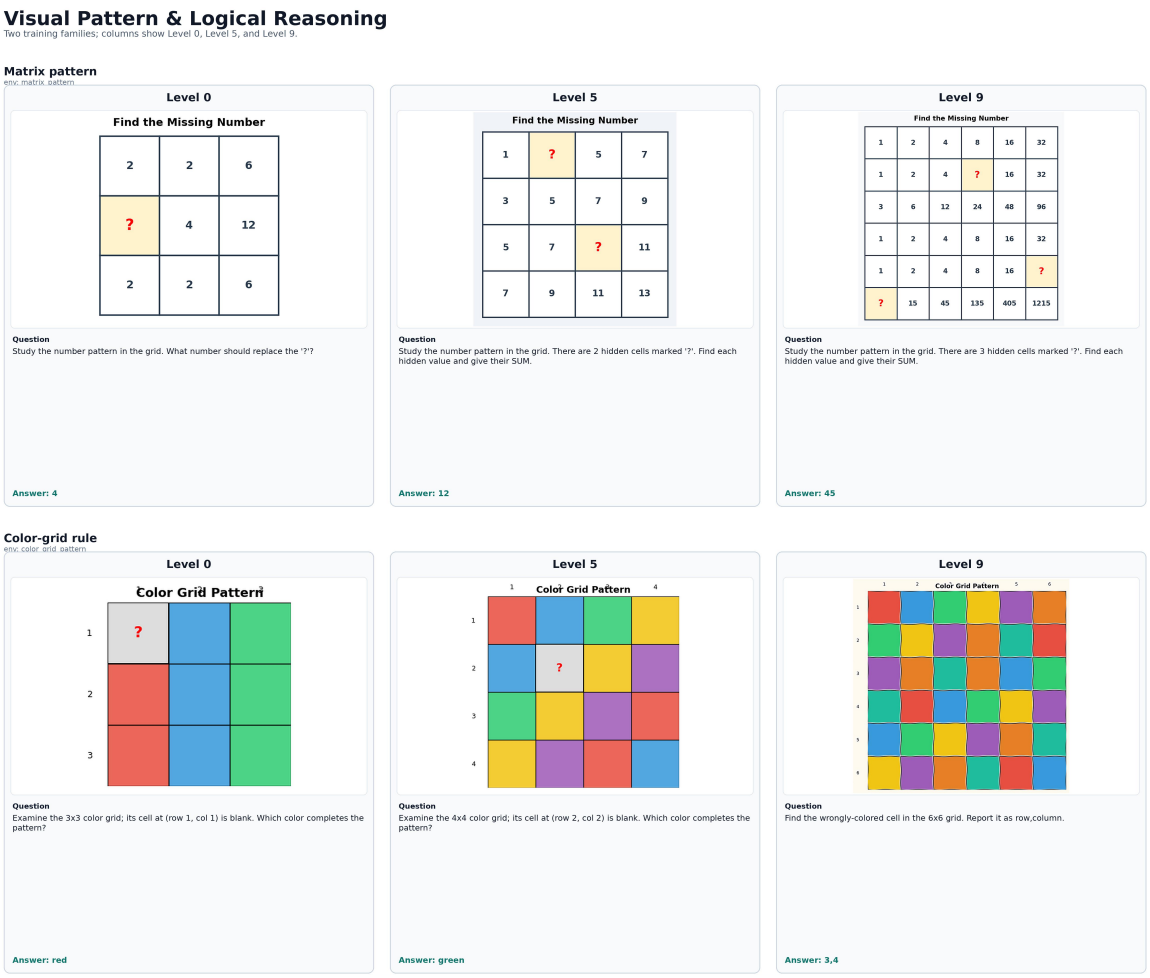


Figure 6: Visual Pattern & Logical Reasoning examples. Rows show matrix pattern completion and color-grid rule induction. Higher levels use larger grids, more symbols or colors, and harder rule violations.

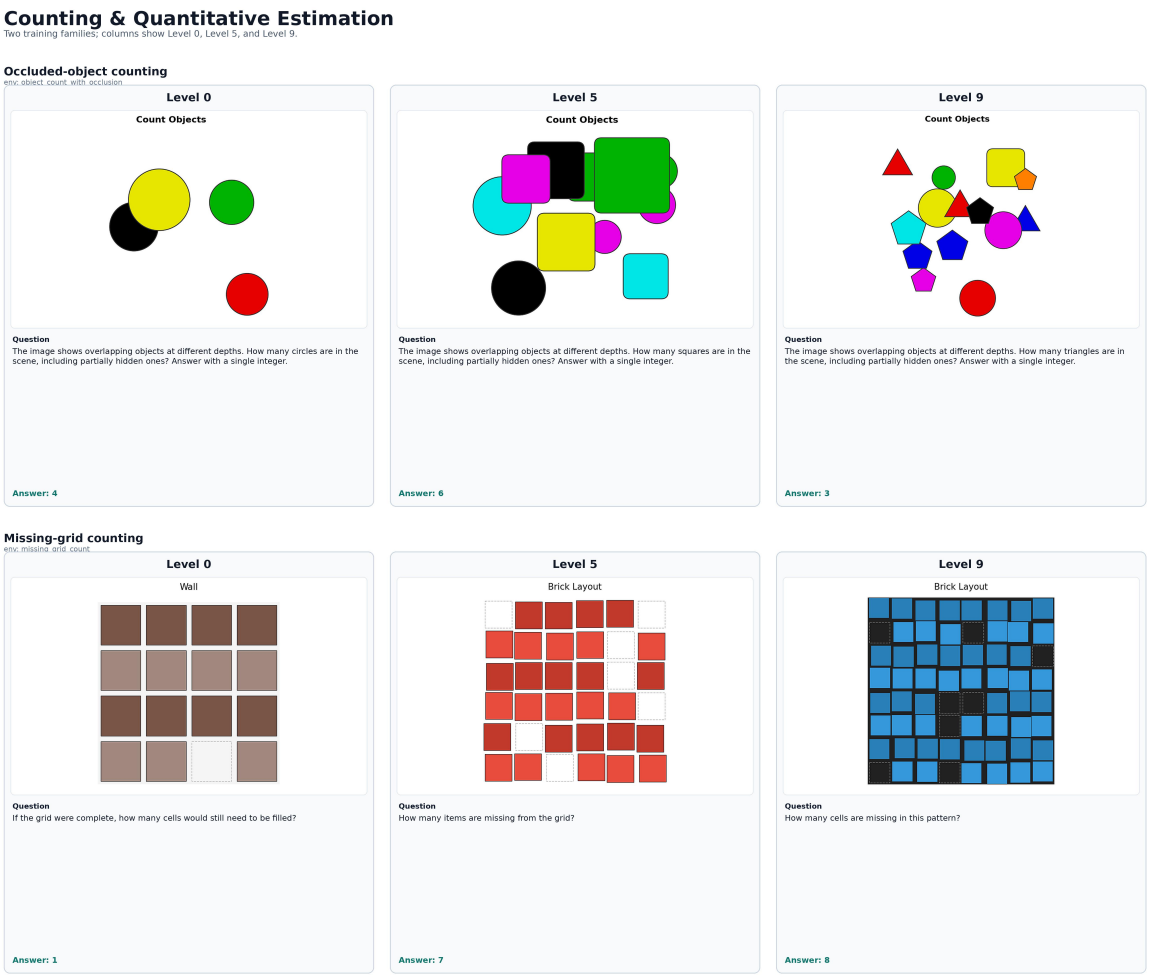


Figure 7: Counting & Quantitative Estimation examples. Rows show occluded-object counting and missing-grid counting. Higher levels increase clutter, occlusion, grid size, and the number of missing or empty cells.

Setting	Qwen3-VL-4B	Qwen2.5-VL-7B	MiMo-VL-7B-SFT
Backbone HF weights	`Qwen3-VL-4B-Instruct`	`Qwen2.5-VL-7B-Instruct`	`MiMo-VL-7B-SFT`
Reported checkpoint step	330	550	550
Train batch size	64	32	32
PPO mini-batch size	64	32	32
PPO micro-batch / GPU	2	2	2
Log-prob micro-batch / GPU	2	1	1
Rollouts per prompt (n)	8	8	8
Rollout temperature	1.0	1.0	1.0
Max prompt length	8192	8192	8192
Max response length	8192	8192	8192
Optimizer	Adam, lr 5×10^{-6}, betas $(0.9, 0.98)$		
Clip ratio (low, high)	(0.2, 0.28)		
Entropy coefficient	0		
KL loss coefficient	0.005 (low-var KL); not added to reward		
Group filter	enabled (metric = accuracy, max gen batches = 10)		
Tensor parallelism	4		
Hardware	4 × H100 80 GB		

Table 7: Full-model training settings for the three backbones reported in the main results. The upper block lists backbone-specific differences (batch sizes scaled to fit 4 H100 GPUs, and the training step at which the reported checkpoint was taken); the lower block is identical across runs. For each backbone the reported checkpoint is selected at convergence on a held-out TRON validation parquet (Appendix D): the 4B run trains for 330 steps and the 7B and MiMo-VL runs train for 550 steps.

Ability	Envs	Step	Comment
Math	131	200	mathematical reasoning
Spatial	111	200	spatial reasoning
Diagram	144	200	diagram and structured visual reasoning
Pattern	104	200	visual pattern and logic
Count	30	100	capped because the bucket is small

Table 8: Ability-specialist bucket sizes and reported checkpoints.

10 generated batches per update; all generated rollouts are scored before filtering and therefore remain available for curriculum promotion. Checkpoints and validation are run every 10 training steps.

The curriculum state is maintained per ability. Promotion uses a minimum accuracy threshold of 0.80, at least eight samples, eight rollouts per prompt, and a difficulty-check batch of 16. The sampler keeps a 0.30 probability of mixing lower-level instances so that training does not immediately collapse onto only the hardest level. Each run reserves a fresh seed block at startup to avoid seed reuse after crashes or restarts.

Each ability has a deterministic validation parquet generated from up to 30 environments from the same bucket. Validation samples levels $\{0, 3, 6, 9\}$ with 20 seeds per level. The auto-restart wrapper runs validation before training on the first attempt to capture the step-0 baseline, then skips repeated baseline validation on restarts and resumes from the latest checkpoint and curriculum snapshot.

Figure 8 shows the resulting training dynamics. The left panel plots validation-accuracy gain over the step-0 baseline; the right panel plots the mean curriculum difficulty across audited levels. All five specialists improve monotonically on their bucket validation set and advance their per-environment curriculum upward as lower levels are mastered. The Count curve stops at step 100 (per the overfitting cap above), while the other specialists run through step 200.

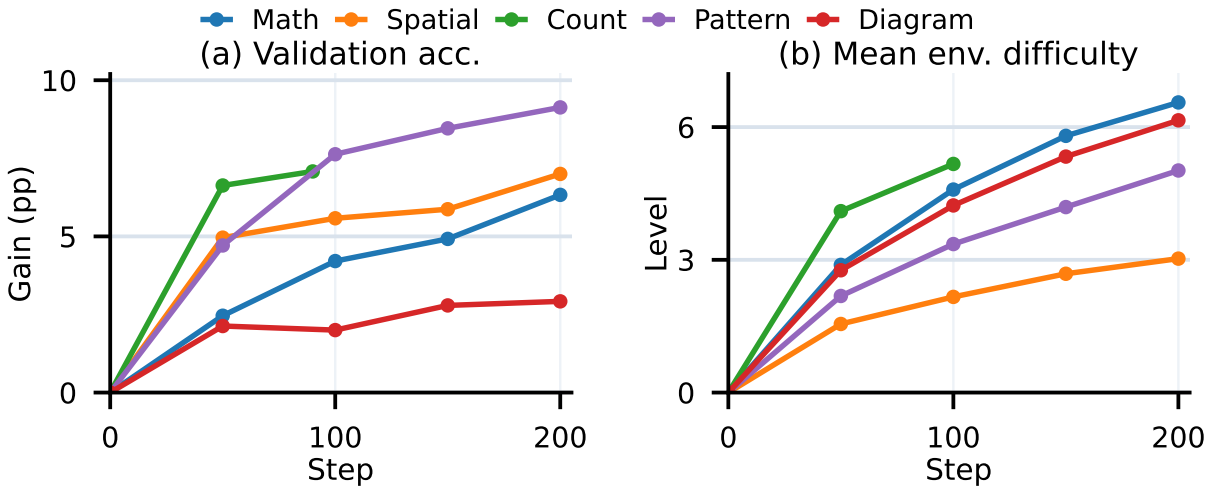


Figure 8: Training dynamics for ability specialists. The left panel shows validation-accuracy gain over step 0; the right panel shows mean curriculum difficulty. Count stops at step 100; other specialists are shown through step 200.

E. Quality Gate Implementation

This appendix spells out the binary predicates used in Section 5.1 (Quality paragraph), summarized in Table 9. A requested probe is counted in $\mathcal{O}_e$ only if the environment generation call completes and returns success; the remaining three gates are evaluated over those successful probes.

The four gates target distinct failure modes. The generation gate catches generator-side exceptions and missing returns. The image gate catches blank, single-color, or saturated renderings through size (≥ 64 px), contrast (grayscale std ≥ 2.0), and foreground-ratio bounds ($[0.001, 0.98]$ relative to the page median). The question/answer gate catches missing or empty fields after normalization. The verifier gate catches verifiers that accept arbitrary strings or reject the canonical correct answer, by checking both a wrapped correct payload (must score 1.0) and a fixed wrong payload (must score 0.0). All thresholds are coarse syntactic bounds chosen to flag obvious failure modes without penalizing environments with legitimately sparse or dense images.

Gate	Predicate used by the audit
Generation	The environment call succeeds without exception and returns a rendered instance.
Image	The rendered image has width and height at least 64 pixels, grayscale standard deviation at least 2.0, and foreground-ratio proxy in $[0.001, 0.98]$. The proxy counts pixels whose grayscale value differs from the page median by more than 10.
Question/answer	The normalized question string and ground-truth answer string are both non-empty.
Verifier	The verifier assigns full accuracy to a wrapped correct answer and zero accuracy to a fixed wrong payload. The audit rotates among the accepted answer wrappers used during training.

Table 9: Implementation-level predicates for the quality gates. These checks are deliberately syntactic and model-free; they catch broken rendering, missing fields, and verifier failures before RL training.

F. Diversity Audit Details

This appendix gives the concrete definitions of every symbol used in Section 5.1.

Setup. For each environment e, audited level $\ell \in \mathcal{L}$, and successfully generated seed k, write $x_{e,\ell,k} = (I_{e,\ell,k}, q_{e,\ell,k}, a_{e,\ell,k})$. The audit extracts three primitives from each probe: a 256-bit image perceptual

hash $\pi(x)$, a number-normalized question template $\tau(x)$, and an answer bucket $\beta(x)$. Let K_ℓ denote the number of successful probes at level ℓ and $T_{e,\ell}$ the set of templates seen. Let $c(y) = \min(1, y)$ be the cap-at-1 function.

Seed signals (h, t, a) at level ℓ. These are the three signals fed to the seed-spread formula in Section 5.1:

$$h_\ell = c\left(\frac{3}{256} \operatorname*{avg}_{i<j} d_H(\pi(x_{e,\ell,i}), \pi(x_{e,\ell,j}))\right), \tag{2}$$

$$t_\ell = \frac{|\{\tau(x_{e,\ell,k})\}_{k=1}^{K_\ell}|}{\max(1, K_\ell)}, \tag{3}$$

$$a_\ell = \frac{H(\{\beta(x_{e,\ell,k})\}_{k=1}^{K_\ell})}{\log_2 \max(2, K_\ell)}, \tag{4}$$

where d_H is Hamming distance and H is Shannon entropy in bits. Intuitively, h_ℓ measures how visually different the rendered images are across seeds (large pairwise pHash distance); t_ℓ measures whether the question wording varies (fraction of distinct number-normalized templates); and a_ℓ measures whether the answers vary (normalized entropy over answer buckets). A signal near 0 indicates seed-collapse on that axis. The seed-spread weights are $(w_h, w_t, w_a) = (0.45,\ 0.25,\ 0.30)$, giving the per-environment seed-diversity score

$$D_s(e) = \operatorname*{avg}_{\ell \in \mathcal{L}} \big(w_h\, h_\ell + w_t\, t_\ell + w_a\, a_\ell\big). \tag{5}$$

The factor 3 in h_ℓ keeps small but real within-generator pHash spread visible after the cap; without it, h_ℓ saturates near zero on most environments.

Level signals (h, j, f) at pair (ℓ, ℓ'). For each adjacent pair $(\ell, \ell') \in \mathcal{A}$, with $\bar{f}_{e,\ell}$ the mean foreground-pixel ratio at level ℓ:

$$h_{\ell,\ell'} = c\left(\frac{1}{120} \operatorname*{avg}_{i,j} d_H(\pi(x_{e,\ell,i}), \pi(x_{e,\ell',j}))\right), \tag{6}$$

$$j_{\ell,\ell'} = 1 - \frac{|T_{e,\ell} \cap T_{e,\ell'}|}{|T_{e,\ell} \cup T_{e,\ell'}|}, \tag{7}$$

$$f_{\ell,\ell'} = c\left(\frac{|\bar{f}_{e,\ell'} - \bar{f}_{e,\ell}|}{0.20}\right). \tag{8}$$

Intuitively, $h_{\ell,\ell'}$ measures whether the rendered images change between two difficulty levels (cross-level pHash distance); $j_{\ell,\ell'}$ measures whether the question templates turn over between levels (Jaccard distance over template sets); and $f_{\ell,\ell'}$ measures whether the image foreground complexity shifts between levels. All three close to 0 would mean the difficulty axis changes only hidden metadata. The level-shift weights are $(w_h, w_j, w_f) = (0.55,\ 0.30,\ 0.15)$, giving the per-environment level-diversity score

$$D_l(e) = \operatorname*{avg}_{(\ell,\ell') \in \mathcal{A}} \big(w_h\, h_{\ell,\ell'} + w_j\, j_{\ell,\ell'} + w_f\, f_{\ell,\ell'}\big). \tag{9}$$

The constants 120 and 0.20 are normalizers chosen so that a typical between-level pHash distance and foreground-ratio change both map into the $[0, 1]$ range before clipping. The w_h in D_l is a different constant from the w_h in D_s; both multiply pHash-based signals, but in different formulas.

Cross-environment predicates $(C_{\text{hash}}, C_{\text{thumb}}, C_{\text{temp}})$. For a pair of environments (e, e'), the audit aggregates their $\ell = 0$ probes into three cross-sample summaries: the mean pHash Hamming distance

$\bar{d}_H(e, e')$, the mean thumbnail-pixel mean absolute error $\bar{m}(e, e')$, and the maximum token-Jaccard similarity $J_{\max}(T_{e,0}, T_{e',0})$ between any pair of normalized L0 prompt templates from the two environments. The three predicates inside the dup formula of Section 5.1 are

$$C_{\text{hash}}(e, e') = [\bar{d}_H(e, e') < 20], \tag{10}$$

$$C_{\text{thumb}}(e, e') = [\bar{m}(e, e') < 8], \tag{11}$$

$$C_{\text{temp}}(e, e') = [J_{\max}(T_{e,0}, T_{e',0}) \geq 0.50]. \tag{12}$$

Thresholds $(20,\ 8,\ 0.50)$ are chosen so that a flag requires visual, pixel, and prompt similarity simultaneously. The per-environment indicator is $D_x(e) = 0$ if there exists $e' \neq e$ with $\text{dup}(e, e') = 1$ and $D_x(e) = 1$ otherwise.

Overall combination. The convex-combination weights in the main-text formula $D(e) = w_s D_s(e) + w_l D_l(e) + w_x D_x(e)$ are $(w_s, w_l, w_x) = (0.55,\ 0.35,\ 0.10)$. The A/B/C/D grade thresholds applied to $D(e)$ are $(0.65,\ 0.50,\ 0.35)$. All constants in this appendix are coarse reporting choices for the diversity histogram in Figure 2; they are not learned and are not used by the RL trainer. The audit additionally writes every raw per-signal value, so flagged environments can be inspected without relying on the aggregate grade alone.

G. Example Environment Implementation

For reference, Listing 1 shows the full source of one TRON environment from the Math bucket. The level ladder (`_level_config`) selects question types as the difficulty level ℓ increases; the generator (`_generate_problem`) samples a clock state, renders it with matplotlib, and returns a `(question, answer, image)` triple. Numerical verification with absolute tolerance 0.001 is inherited from the base class `StandaloneVisualEnv`.

Listing 1: Full source of `clock_angle_qa.py`.

```python
"""
Clock angle QA -- analog clock showing a time.
Questions: angle between hands, time shown, angle after N minutes, overlap count.
"""
import math
from typing import Dict, Optional, Tuple
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
from PIL import Image
from .standalone_base import StandaloneVisualEnv

class ClockAngleQA(StandaloneVisualEnv):
    ALLOW_ROTATION = False  # orientation-sensitive: disable rotation augmentation
    BENCHMARK_NUM_TOLERANCE_ABS = 0.001
    ENV_NAME = "clock_angle"

    def _hand_angle(self, h, m):
        """Return (hour_deg, minute_deg) measured clockwise from 12."""
        min_deg = 6 * m
        hour_deg = 30 * (h % 12) + 0.5 * m
        return hour_deg, min_deg

    def _angle_between(self, h, m):
        hd, md = self._hand_angle(h, m)
        diff = abs(hd - md)
        return min(diff, 360 - diff)

    def _level_config(self, level: int) -> Dict:
        level = max(0, min(level, 9))
        if level <= 2:
            return {"qtypes": ["read_time", "minute_hand_angle"]}
        if level <= 5:
```

```
            return {"qtypes": ["read_time", "angle_between",
                               "minute_hand_angle"]}
        if level <= 7:
            return {"qtypes": ["angle_between", "angle_after_n"]}
        return {"qtypes": ["angle_after_n", "overlap_count"]}

    def _generate_problem(self, seed: int, parameter: Dict) -> Optional[Tuple[str, str, Image.Image]]:
        rng = self._rng
        level = int(parameter.get("level", 0))
        cfg = self._level_config(level)
        style = self._random_style()
        qtype = parameter.get("question_type", rng.choice(cfg["qtypes"]))

        h = rng.randint(1, 12)
        m = rng.choice([0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55])

        hd, md = self._hand_angle(h, m)

        sc = style["figsize_scale"]
        fig, ax = plt.subplots(figsize=(5 * sc, 5 * sc))
        fig.patch.set_facecolor(style["bg_color"])
        ax.set_facecolor(style["bg_color"])

        clock_face = plt.Circle((0, 0), 1.05, fc="white", ec=style["geo_line_color"],
                                linewidth=style["line_width"] + 1)
        ax.add_patch(clock_face)

        # Hour markers
        for i in range(1, 13):
            ang = math.radians(90 - 30 * i)
            ax.text(0.85 * math.cos(ang), 0.85 * math.sin(ang), str(i),
                    ha="center", va="center", fontsize=style["font_size_base"],
                    fontweight="bold", fontfamily=style["font_family"])
            ax.plot([0.95 * math.cos(ang), 1.0 * math.cos(ang)],
                    [0.95 * math.sin(ang), 1.0 * math.sin(ang)],
                    color=style["geo_line_color"], linewidth=1.5)

        # Hour hand
        h_ang = math.radians(90 - hd)
        ax.plot([0, 0.5 * math.cos(h_ang)], [0, 0.5 * math.sin(h_ang)],
                color=style["palette"][0], linewidth=style["line_width"] + 2,
                solid_capstyle="round")

        # Minute hand
        m_ang = math.radians(90 - md)
        ax.plot([0, 0.75 * math.cos(m_ang)], [0, 0.75 * math.sin(m_ang)],
                color=style["palette"][1], linewidth=style["line_width"] + 0.5,
                solid_capstyle="round")

        ax.plot(0, 0, "o", color=style["geo_line_color"], markersize=5, zorder=5)
        ax.set_xlim(-1.3, 1.3); ax.set_ylim(-1.3, 1.3)
        ax.set_aspect("equal"); ax.axis("off")
        ax.set_title("Analog Clock", fontsize=style["font_size_base"] + 2,
                     fontweight="bold")
        img = self.fig_to_pil(fig, dpi=style["dpi"])

        time_str = f"{h}:{m:02d}"
        angle = round(self._angle_between(h, m), 1)

        if qtype == "angle_between":
            q = ("For the time shown on the clock, what is the angle (in "
                 "degrees) between the hour and minute hands?")
            return q, str(angle), img
        elif qtype == "read_time":
            q = "What time is shown on the clock? Answer in H:MM format."
            return q, time_str, img
        elif qtype == "angle_after_n":
            dm = rng.choice([15, 30, 45, 60])
            new_m = (m + dm) % 60
            new_h = h + (m + dm) // 60
            new_angle = round(self._angle_between(new_h, new_m), 1)
            q = (f"Starting from the time shown on the clock, what will the "
                 f"angle between the hands be after {dm} minutes? Round to "
```

```
                f"1 decimal.")
            return q, str(new_angle), img
        elif qtype == "overlap_count":
            n_hours = rng.choice([6, 12, 24])
            overlaps = round(n_hours * 11 / 12)
            if n_hours == 12:   overlaps = 11
            elif n_hours == 24: overlaps = 22
            elif n_hours == 6:  overlaps = 5
            q = f"How many times do the clock hands overlap in {n_hours} hours?"
            return q, str(overlaps), img
        elif qtype == "minute_hand_angle":
            q = ("For the time shown on the clock, how many degrees has the "
                 "minute hand moved from 12? Answer as a number.")
            return q, str(round(md, 1)), img
        return None
```